Technical prep

Everything the mentor may ask about how VoteChain is built, with the file to open for each part.

The modules

VoteChain has **5 functional modules**, built on **3 technical layers**. If the mentor asks "how many modules", give the five functional ones; if he asks about architecture, give the three layers.

5 functional modules (what the system does)

#	Module	What it does	Main files
1	Organization portal	Register an organization; create, edit, publish, close and delete elections; add candidates; upload voters; send reminders; publish results; audit log	`client/src/pages/org/`, `server/src/routes/org.js`
2	Voter portal	Sign in with an email code; see invited elections; read candidate profiles; vote once; get a receipt; voting history	`client/src/pages/voter/`, `server/src/routes/voter.js`

#	Module	What it does	Main files
3	Blockchain layer	Store elections on-chain, check every vote, block duplicates, keep results sealed, return a transaction hash	`contracts/contracts/VoteChain.sol`, `server/src/chain/index.js`
4	Verification module	Public results, the list of vote transactions, browser-side integrity checks, receipt lookup	`client/src/pages/public/VerifyElection.jsx`, `client/src/lib/chain.js`, `server/src/routes/public.js`
5	Admin dashboard	Approve or suspend organizations, see all elections, system health, relayer balance, rejected transactions	`client/src/pages/admin/`, `server/src/routes/admin.js`

Authentication (sign-in, sessions, roles) runs across all five; its code is `server/src/routes/auth.js` and `server/src/middleware/auth.js`.

3 technical layers (how it is built)

Layer	Folder	Size
Frontend (React web app)	`client/`	21 screens
Backend (Express API + MongoDB)	`server/`	42 API endpoints, 7 database collections
Smart contract (Solidity)	`contracts/`	1 contract, 4 state-changing functions, 4 read functions

Tech stack, and why

This is the MERN stack (MongoDB, Express, React, Node) plus Solidity for the blockchain. Every choice has a one-line reason you can give.

Technology	Version	Used for	Why we chose it
React	18	The web app's screens	Component-based, the most widely used UI library, fits the MERN stack
Vite	5	Builds and serves the React app	Much faster start and reload than Create React App
Tailwind CSS	4	Styling	Style directly in the markup; consistent design with less CSS
React Router	6	Page navigation (`/vote` , `/org` , ...)	Standard routing library for React single-page apps
Node.js	20	Runs the backend	JavaScript on both frontend and backend; huge library ecosystem
Express	4	The REST API	Small, simple, the standard Node web framework
MongoDB + Mongoose	7 / 8	Database	Flexible documents suit elections with varying candidate lists; free cloud hosting on Atlas
Solidity	0.8.24	The smart contract	The main language for Ethereum; 0.8 has built-in overflow checks
Hardhat	2	Compile, test and deploy the contract; local blockchain	Runs a local Ethereum network on the laptop and has great testing tools
OpenZeppelin	5	`AccessControl` and `MerkleProof` libraries	Audited, industry-standard contract code instead of writing our own security

Technology	Version	Used for	Why we chose it
ethers.js	6	Talking to the blockchain from Node and the browser	Smaller and cleaner than web3.js
JWT (jsonwebtoken)	9	Sign-in sessions	Stateless tokens; the server doesn't need to store sessions
bcryptjs	2	Password hashing	Slow hashing on purpose, so stolen hashes are hard to crack
Nodemailer	6	Sending sign-in codes and invites	Works with any SMTP provider
zod	3	Validating request data	Clear rules and error messages for every input
helmet, express-rate-limit	8 / 7	Security headers, limiting login attempts	Standard protections for Express apps
Ethereum Sepolia	testnet	The public blockchain we deploy to	Behaves like the real Ethereum, but its ETH is free from faucets

Why Ethereum and not another blockchain? It's the most established smart-contract platform, with the best tools and documentation. The same contract also runs unchanged on cheaper Ethereum-compatible networks such as Polygon or Base.

Smart contract: `contracts/contracts/VoteChain.sol`

One contract, about 170 lines. It inherits OpenZeppelin's `AccessControl` for roles and uses `MerkleProof` to check voter eligibility.

What it stores

Storage	Type	Holds
`_elections`	`mapping(uint256 => Election)`	Every election, by ID
`_tally`	`mapping(uint256 => mapping(uint8 => uint32))`	Votes per candidate, per election
`_voted`	`mapping(uint256 => mapping(bytes32 => bool))`	Which voter tokens have voted, per election

The `Election` struct holds: `orgId`, `voterRoot` (Merkle root of the voter list), `detailsHash`, `startTime`, `endTime`, `voteCount`, `candidateCount`, `closed`, `exists`.

Roles

- `DEFAULT_ADMIN_ROLE`: the deployer's wallet. It can grant or revoke roles.
- `RELAYER_ROLE`: our server's wallet. Only it can create elections and cast votes.

Functions

Function	Who can call	What it checks	What it does
`createElection(id, orgId, start, end, candidateCount, voterRoot, detailsHash)`	Relayer	ID not used before; end after start and in the future; 2–50 candidates; voter root not empty	Saves the election, emits `ElectionCreated`
`updateElection(id, start, end, candidateCount, voterRoot, detailsHash)`	Relayer	Election exists, not closed, voting not started	Updates settings, emits `ElectionUpdated`
`castVote(id, voterToken, candidate, proof)`	Relayer	Election open right now; candidate valid; token hasn't voted; Merkle proof valid	Marks the token as voted, adds 1 to the candidate and the total, emits `VoteCast`

Function	Who can call	What it checks	What it does
`closeElection(id)`	Relayer any time, anyone after the end time	Exists, not already closed	Sets `closed`, emits `ElectionClosed`
`getElection(id)`	Anyone (read)	Exists	Returns the struct
`hasVoted(id, token)`	Anyone (read)	None	true or false
`isFinal(id)`	Anyone (read)	Exists	true if closed or the end time has passed
`getResults(id)`	Anyone (read)	`isFinal` must be true, otherwise it reverts	Returns the votes for each candidate

Events (the public log)

`ElectionCreated`, `ElectionUpdated`, `VoteCast(electionId, voterToken, voteNumber)`, `ElectionClosed`. **`VoteCast` deliberately doesn't include the candidate**, so the event log alone doesn't show who voted for whom.

Custom errors (why a transaction can be rejected)

`ElectionExists`, `ElectionNotFound`, `InvalidWindow`, `InvalidCandidateCount`, `InvalidVoterRoot`, `AlreadyStarted`, `ElectionNotActive`, `ElectionAlreadyClosed`, `ElectionStillRunning`, `InvalidCandidate`, `AlreadyVoted`, `InvalidMerkleProof`. Custom errors cost less gas than text error messages, and the server translates each one into a readable message (`FRIENDLY` in `server/src/chain/index.js`).

How the Merkle check works in the code

The leaf is `keccak256(bytes.concat(keccak256(abi.encode(voterToken))))`, hashed twice in the format OpenZeppelin's Merkle-tree library uses. `MerkleProof.verifyCalldata(proof, voterRoot, leaf)` returns true only if this token was in the voter list when the root was made.

Backend: `server/`

Folders

Path	Role
`src/index.js`	Starts everything: connects the database, checks the chain, starts the server, seeds demo data, and closes expired elections every minute
`src/app.js`	Builds the Express app: security headers, CORS, JSON parsing, routes, error handler, and serving the built web app
`src/config.js`	Reads all settings from environment variables (`.env`)
`src/db.js`	Connects to MongoDB (Atlas in production, an embedded database locally)
`src/models/index.js`	The 7 database models (Mongoose schemas)
`src/routes/`	The 42 endpoints, one file per role
`src/middleware/auth.js`	JWT creation and checking, and role checks
`src/services/elections.js`	Core logic: publish, update, close, vote, results, turnout
`src/services/serialize.js`	Shapes database records into the JSON the web app expects; hides totals until final
`src/services/mailer.js`	Emails: sign-in code, invite, reminder, results
`src/chain/index.js`	The relayer wallet and every call to the contract
`src/lib/crypto.js`	Voter tokens, Merkle tree, details hash, receipt codes, sign-in codes
`src/lib/http.js`	Error classes and input validation helpers

What happens to one request

Request → `helmet` (security headers) → `cors` → `express.json` → `cookieParser` → the route's rate limiter → `requireAuth(role)` checks the JWT and role → `zod` validates the input → the service function does the work → `serialize` shapes the reply → JSON response. Any error goes to the error handler in `app.js`, which returns a clean message and status code.

The 42 endpoints

Group	Base path	Who	Endpoints
Auth	`/api/auth`	Anyone / signed in	`POST register-org`, `POST login`, `POST otp/request`, `POST otp/verify`, `POST refresh`, `POST logout`, `POST logout-all`, `GET me`, `PUT me`, `PUT me/password`
Organizer	`/api`	Org admin	`GET org`, `PUT org`, `GET org/overview`, `GET elections`, `POST elections`, `GET elections/:id`, `PUT elections/:id`, `DELETE elections/:id`, `POST elections/:id/publish`, `POST elections/:id/close`, `POST elections/:id/publish-results`, `GET elections/:id/voters`, `GET elections/:id/voters.csv`, `POST elections/:id/voters`, `DELETE elections/:id/voters/:inviteId`, `POST elections/:id/remind`, `GET elections/:id/audit`
Voter	`/api/voter`	Voter	`GET elections`, `GET elections/:id`, `POST elections/:id/vote`, `GET votes`, `GET receipts/:id`
Public	`/api/public`	Anyone	`GET chain`, `GET elections`, `GET elections/:id`, `GET elections/:id/ledger`, `GET receipts/:code`
Admin	`/api/admin`	Platform admin	`GET overview`, `GET organizations`, `PATCH organizations/:id`, `GET elections`, `GET health`

Plus `GET /api/health`, a simple liveness check that Render uses.

The 7 database collections

Collection	Key fields	Notes
`users`	name, email (unique), passwordHash, role, orgId, memberId, otp {hash, expiresAt, attempts}, tokenVersion, prefs	role is `platform_admin`, `org_admin` or `voter`. Passwords and codes are stored only as hashes
`organizations`	name, type, description, website, allowedDomain, members, status, admins	status is `pending`, `active` or `suspended`
`elections`	orgId, title, description, category, startAt, endAt, state, candidates[], eligible, votesCast, tally, chain {electionId, voterRoot, detailsHash, txHash, block, closeTxHash}, closedAt, publishedAt	state: `draft` → `active` → `closed` → `published`. Candidates are stored inside the election
`invites`	electionId, orgId, email, name, memberId, voterToken, status, votedAt	The voter list. Unique per election + email. status: `invited`, `pending`, `voted`
`receipts`	electionId, userId, code, voterToken, txHash, block, castAt	Proof of voting. **Never stores the candidate**
`activities`	orgId, electionId, kind, text, actor, at	The audit log and activity feed
`failedtxes`	electionId, action, reason, txHash, at	Transactions the contract rejected, shown on System health

Frontend: `client/`

A React single-page app: the browser loads one page, and React swaps screens without full reloads.

Folders

Path	Role
`src/main.jsx`	Entry point: wraps the app in the router and three providers (auth, chain info, toasts)

Path	Role
`src/App.jsx`	All routes: which URL shows which page and layout
`src/layouts/`	`PublicLayout` (top bar and footer) and `DashboardLayout` (sidebar, role check, sign-out menu)
`src/pages/public/`	Home, sign in, register, public results, election results, receipt check, 404
`src/pages/org/`	Overview, elections list, election editor (the 4-step wizard), election detail, settings
`src/pages/voter/`	My elections, election, ballot, receipt, history, profile
`src/pages/admin/`	Overview, organizations, elections, system health
`src/components/ui.jsx`	Shared building blocks: Button, Card, Badge, Modal, Tabs, Table, inputs, loaders
`src/components/election.jsx`	Election card, results bars, sealed-results notice, turnout chart (hand-drawn SVG, no chart library)
`src/store/auth.jsx`	Who is signed in; login, email-code, register and logout functions
`src/lib/api.js`	The single function every page uses to call the API; refreshes the session automatically
`src/lib/useApi.js`	A hook that loads data and handles loading and error states; can auto-refresh
`src/lib/chain.js`	Browser-side blockchain checks for the public results page (uses ethers.js)
`src/index.css`	The colour theme (light and dark) as CSS variables

How a page gets its data

1. The page calls `useApi('/voter/elections')`.
2. `api.js` sends the request with the access token in the `Authorization` header.
3. If the token has expired (after 15 minutes), the server answers 401. `api.js` then calls `/api/auth/refresh`, which uses the httpOnly cookie, gets a new token, and retries the request. The user never notices.
4. While waiting, the page shows a spinner; on error, a message with a Try again button.

How role protection works on the frontend

`DashboardLayout` takes a role (`org_admin` , `voter` or `platform_admin`). If nobody is signed in it redirects to `/login` ; if the signed-in user has a different role, it sends them to their own dashboard. **This is only for convenience.** The real protection is on the server, where every endpoint checks the role again.

Why the access token isn't in localStorage

Anything in localStorage can be read by any script that runs on the page. So the short-lived access token lives only in memory, and the long-lived refresh token is in an httpOnly cookie that JavaScript can't read at all.

Where to show the database and the blockchain

There are two kinds of data. Show both, and point out the difference: votes are on the blockchain, everything else is in MongoDB.

Option 1: MongoDB Compass (best for the demo)

Compass is MongoDB's free desktop app for browsing a database.

1. Download and install **MongoDB Compass** from mongodb.com/products/compass.
2. In `D:\votechain` , run `npm run dev` . In the terminal, look for this line: `[db] Browse it in MongoDB Compass: mongodb://127.0.0.1:27018/votechain`
3. In Compass, click **Add new connection**, paste `mongodb://127.0.0.1:27018/votechain` , then **Connect**.
4. Open the **votechain** database. You'll see the 7 collections: users, organizations, elections, invites, receipts, activities, failedtxes.

The local database lives in memory, so it resets every time `npm run dev` restarts. Keep the app running during the demo.

Alternative: install the **MongoDB for VS Code** extension and connect with the same address, to browse the data inside VS Code.

Things worth pointing at in Compass:

- `users` → `passwordHash` is a bcrypt hash, never the real password.
- `elections` → the `chain` field holds the on-chain election ID, voter root, details hash and the transaction that created it.
- `invites` → each voter has a `voterToken` , the anonymous code that goes on the blockchain instead of their email.

- `receipts` → there is **no candidate field**. The database doesn't know who anyone voted for.
- `activities` → the audit log of everything that happened.

Option 2: inside the app (no install needed)

- **Admin → Organizations / Elections / System health**: all organizations, all elections, database and blockchain status, rejected transactions.
- **Organizer → an election → Voters tab**: the voter list with who has voted (never how); **Export CSV** downloads it.
- **Organizer → an election → Audit log tab**: every action with time and person.

Option 3: after deployment, the Atlas website

On mongodb.com/atlas, open the cluster and click **Browse Collections**. Same 7 collections, live from the deployed site, viewable from any browser.

Showing the blockchain data

- **Locally:** the `npm run dev` terminal shows every transaction the local blockchain mines, with lines starting `[chain]`. The **public results page** of any election lists its vote transactions with hashes and block numbers, and its integrity checks read the contract directly.
- **After deploying to Sepolia:** every hash in the app links to **Sepolia Etherscan** (sepolia.etherscan.io), a public blockchain explorer. Open the contract address there to show every transaction ever sent to it. Nobody can edit that list, including us.

Key terms in simple words

Term	Simple meaning	Where it appears in VoteChain
Blockchain	A shared record book copied across thousands of computers. New pages can be added, but old pages can never be changed	Where votes are stored
Block	One page of that record book, holding a batch of transactions	Every receipt shows the block number its vote went into

Term	Simple meaning	Where it appears in VoteChain
Transaction	One action written to the blockchain, signed by a wallet	Creating an election, each vote, closing
Transaction hash	The unique ID of a transaction (0x… plus 64 characters)	Shown on receipts; used to look a vote up
Smart contract	A program stored on the blockchain that runs exactly as written; nobody can change its rules after deployment	`VoteChain.sol`
Ethereum	The most widely used blockchain for smart contracts	Our platform
Sepolia testnet	A practice copy of Ethereum, with free test ETH	Where we deploy
Gas	The fee paid in ETH to run a transaction	Our relayer pays it for every vote
Wallet / private key	An account on the blockchain; the private key is its password and signs transactions	The relayer wallet on our server
Relayer	A server wallet that sends transactions on behalf of users	Lets voters vote without any crypto
RPC URL	The web address of a blockchain node that the app talks to	Alchemy for the server; a public node for the browser checks
ABI	The list of a contract's functions, so code knows how to call them	`deployment.sepolia.json`, generated by the deploy script
Event	A log entry a contract writes when something happens; cheap to store and easy to search	`VoteCast`, `ElectionCreated`

Term	Simple meaning	Where it appears in VoteChain
Hash (keccak256, SHA-256)	A fingerprint of data: same input, same output; any change gives a completely different fingerprint; can't be reversed	Details hash, voter tokens, sign-in codes
HMAC	A hash mixed with a secret key, so only someone with the key can produce it	Making voter tokens from emails
Merkle tree / root / proof	All voters' tokens combined, pair by pair, into one fingerprint (the root). A proof is the short path showing that one token is inside it	Checking voter eligibility on-chain without storing the list
JWT	A signed token that proves who you are, so the server needn't look you up in a session table	Access token after sign-in
bcrypt	A deliberately slow password-hashing method	Storing passwords
OTP	A one-time password, here a 6-digit code sent by email	Voter sign-in
httpOnly cookie	A cookie the page's JavaScript can't read, so it's safe from script attacks	Holds the refresh token
REST API	A set of URLs the frontend calls to read or change data	The 42 endpoints
Single-page app (SPA)	A website that loads once, then swaps screens without full reloads	The React frontend

Testing

34 automated tests, all passing. Run them all from `D:\votechain` with `npm test`.

Smart contract: 17 tests (`contracts/test/VoteChain.test.js`)

Written with Hardhat, Mocha and Chai. Each test deploys a fresh contract on a temporary blockchain and uses Hardhat's time helpers to jump forward to the voting window or past the end.

- **createElection:** stores the election; rejects a duplicate ID; rejects a bad window, too few or too many candidates, an empty voter root; only the relayer can create
- **updateElection:** allowed before voting opens; blocked after
- **castVote:** records a valid vote without the candidate in the event; rejects a second vote; rejects voters not on the list; rejects an invalid candidate; rejects votes before the start and after the end; rejects an unknown election; only the relayer can vote
- **Closing and results:** results sealed while open; relayer can close early and the tally is revealed; others can close only after the end; results reveal automatically at the end time

Run only these: `cd contracts` then `npm test`.

API: 17 tests (`server/test/api.test.js`)

Written with Node's built-in test runner and Supertest, against an in-memory database. The blockchain is replaced by a **fake chain** that follows the same rules as the contract, so these tests run in seconds without a blockchain.

- **Onboarding:** registering creates a pending organization and an httpOnly cookie; duplicate email rejected; publishing blocked until admin approval; role checks on admin routes
- **Election lifecycle:** draft → add voters → publish on-chain; locked after voting starts; voter signs in with an email code; strangers can't get a code; vote returns a receipt and a second vote is refused; results sealed while open; **the receipt never contains the candidate**; can't vote in elections you aren't invited to; close early, tally revealed, late vote refused, results published; only drafts can be deleted
- **Sessions:** refresh from the cookie; wrong password and wrong code rejected
- **Consistency:** eligible voters and votes cast add up

Run only these: `cd server` then `npm test`.

Manual end-to-end testing

The full flow was also tested in the browser on a real local blockchain: publish an election, vote, check the receipt, close early, and confirm that all five integrity checks pass on the public page. That testing found and fixed two real bugs:

1. The local blockchain's clock ran about 6 seconds behind the server's, so elections set to start "now" opened in the chain's future and the first vote was rejected. **Fix:** elections that start immediately now take their start time from the blockchain.
2. A leftover server from an earlier run kept answering after the blockchain was restarted, so votes failed with "election not found on-chain". **Fix:** the start script now refuses to run if old processes are still holding the ports.

Mentioning these in the review shows real testing, not just happy-path demos.

Question bank

Blockchain and smart contract (Member 3)

Question	Answer
Why blockchain for voting?	A normal database can be edited by whoever controls it. The blockchain can't be edited once a vote is written, and anyone can inspect it, so the count is trustworthy without trusting us.
What exactly is stored on-chain?	Each election's window, number of candidates, the Merkle root of the voter list, a hash of the details, which voter tokens have voted, and the vote totals. No names, emails or photos.
How do you stop double voting?	`_voted[electionId][voterToken]` is set to true on the first vote; a second `castVote` with the same token reverts with `AlreadyVoted`.
How does the contract know who's allowed to vote without storing the list?	A Merkle root of all voter tokens is stored at creation. Each vote carries a Merkle proof; `MerkleProof.verifyCalldata` checks it against the root.
Why can't the organizer see who is winning during voting?	`getResults` reverts with `ElectionStillRunning` until the election is closed or its end time passes.
Who can call `castVote`?	Only addresses with `RELAYER_ROLE`, i.e. our server's wallet. OpenZeppelin's `onlyRole` modifier enforces it.

Question	Answer
What is `onlyRole`?	A modifier from AccessControl: it runs before the function and reverts if the caller doesn't have the role.
Why custom errors instead of `require` messages?	They cost less gas and the server can match them by name to show friendly messages.
Why `uint32` and `uint8` instead of `uint256`?	They fit 4 billion votes and 255 candidates, and smaller types can be packed into fewer storage slots, which saves gas.
Can the contract be changed after deployment?	No. Its code is permanent. A new version would be a new deployment at a new address.
What does deploying cost on Sepolia?	About 0.005 test ETH, which is free from faucets. Each vote costs about 0.0001.

Backend and database (Member 2)

Question	Answer
Why MongoDB and not SQL?	Elections hold varying lists of candidates and manifesto points, which fit naturally as nested documents. It's also part of the MERN stack, and Atlas hosts it for free.
How are passwords stored?	Hashed with bcrypt at cost 12. We never store or log the real password.
How does email-code sign-in work?	A random 6-digit code is generated, only its SHA-256 hash is stored, and it expires in 10 minutes. After 5 wrong attempts it's blocked.
How do you know a request is allowed?	`requireAuth(role)` middleware checks the JWT signature and expiry, loads the user, and compares their role.
What stops two clicks submitting two votes at once?	The invite is atomically switched from `invited` to `pending` with `findOneAndUpdate` before sending the transaction; the second request finds no `invited` row and is refused.

Question	Answer
How is input validated?	Every endpoint parses the body with a zod schema; bad input gets a 400 with a clear message.
What happens when an election's time ends?	A job runs every minute (`closeExpired`), marks it closed, and caches the final totals from the contract.
How do you prevent brute-force login?	`express-rate-limit` : 30 attempts per 15 minutes on sign-in routes, and 20 votes per minute.

Frontend (Member 1)

Question	Answer
Why React?	Reusable components (one Button, Card, Badge used everywhere) and the largest ecosystem. Part of MERN.
How do pages talk to the backend?	Through `lib/api.js` and the `useApi` hook, which handle tokens, loading, errors and silent session refresh.
How does the app remember a logged-in user after refresh?	On load, `AuthProvider` calls `/api/auth/refresh` ; the httpOnly cookie proves the session, and a new access token comes back.
How did you make it responsive?	Tailwind's breakpoint classes (`sm:` , `md:` , `lg:`); every page was checked at phone width.
How does dark mode work?	Colours are CSS variables defined twice, for light and dark. A `dark` class on the page switches them, and the choice is saved in localStorage.
What's on the ballot page?	Candidate cards as radio buttons, a confirm dialog, then progress steps while the vote is recorded, then a redirect to the receipt.

Integration and security (Member 4)

Question	Answer
Why don't voters need MetaMask?	The relayer wallet on our server signs and pays for every transaction. Voters only use email.
How is the voter's identity hidden on-chain?	The server computes an anonymous token, HMAC-SHA256 of election ID + email with a secret key, then keccak256. Only the token goes on-chain.
How does the public page verify results?	`lib/chain.js` uses ethers.js in the browser to read the contract through a public RPC, then compares the vote count, details hash, voter root, candidate count and totals with what the API shows.
What if two votes arrive at the same moment?	Transactions are queued and sent one at a time, so the wallet's nonce (its transaction counter) never clashes. ethers' `NonceManager` tracks it.
Where are secrets kept?	In environment variables (`.env` locally, the Render dashboard in production). `.env` files are in `.gitignore`.
What is the Content Security Policy for?	It tells the browser which sources may load scripts, styles, fonts and data, which blocks injected scripts.

Project and design (Member 5)

Question	Answer
How many modules?	Five: organization portal, voter portal, blockchain layer, verification module, admin dashboard.
What's your architecture?	Three layers: React frontend, Express + MongoDB backend, Solidity contract on Ethereum. One Render service serves both the frontend and the API on a single domain.
What's the development model?	Incremental: requirements and PRD, UI prototype on sample data, contract with tests, backend with tests, integration, then end-to-end testing.

Question	Answer
What's left to do?	Deploy to Sepolia and Render, run a trial election with students, and finish the SRS, UML diagrams and user manual.
What would you add next?	Voter-signed ballots (so the server can't vote for anyone), secret-ballot cryptography (commit-reveal or zero-knowledge proofs), ranked-choice voting, and a mobile app.
What are the limitations?	The relayer is trusted to submit votes; the chosen candidate is visible in the transaction data next to an anonymous token; and it hasn't had a professional security audit.